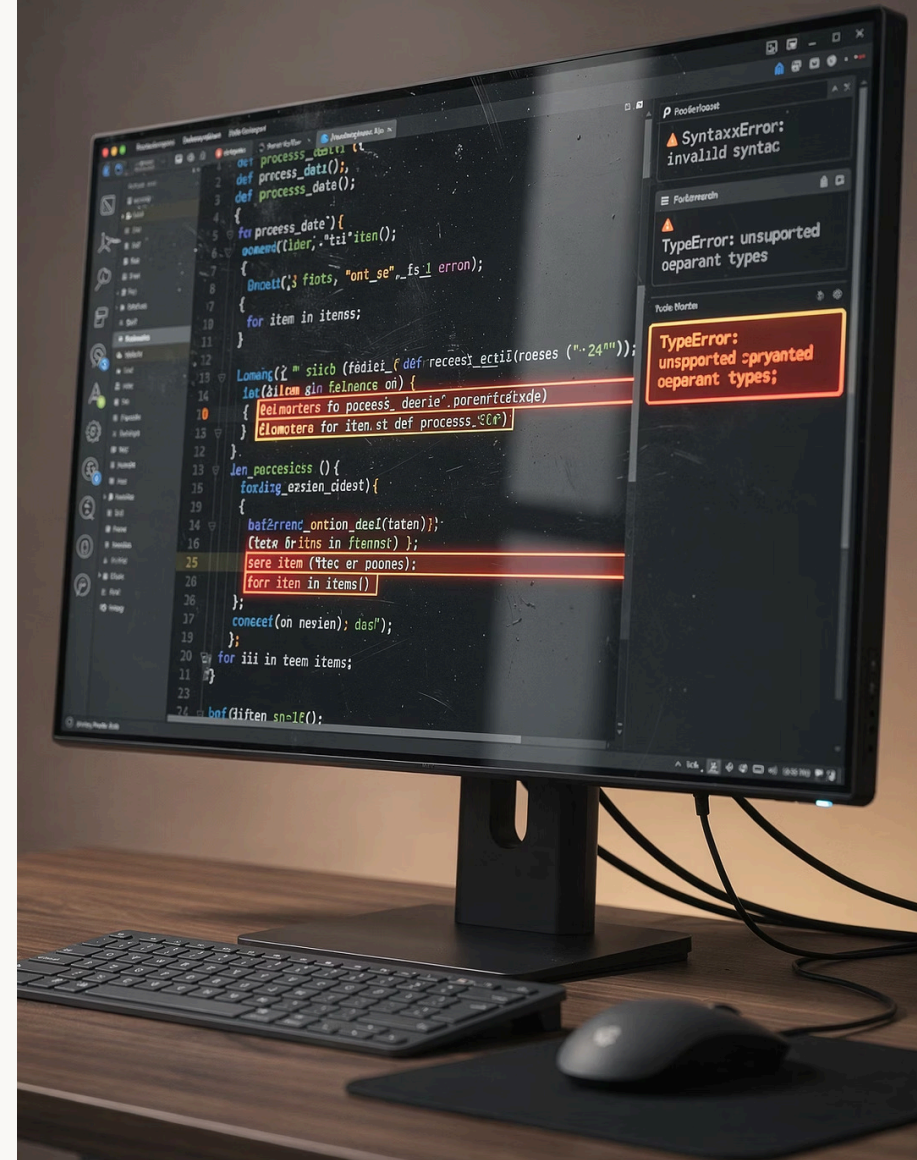


# Excepciones Personalizadas en Python



# El punto de partida: validar con if

Imagina que tienes que registrar la edad de un usuario. Una primera solución natural es validar con if y mostrar un mensaje:

```
edad = int(input("Introduce tu edad: "))
```

```
if edad < 0 or edad > 120:
```

```
    print("Error: la edad no es válida.")
```

```
else:
```

```
    print(f"Edad registrada: {edad}")
```

- ❗ Esto funciona para casos simples... pero ¿qué pasa cuando el programa crece y necesitas gestionar ese error desde otra parte del código?



# Las limitaciones del print

## ¿Qué problema tiene este enfoque?

- El error se muestra, pero el programa **sigue ejecutándose** como si nada hubiera pasado
- No puedes **capturar** ese error en otro punto del código
- No distingues **qué tipo** de error ocurrió
- En proyectos grandes, el código se vuelve difícil de mantener

## Lo que necesitamos

Un mecanismo que **interrumpa el flujo**, identifique el error con un nombre claro y permita **reaccionar** desde cualquier parte del programa.

- ❏ ¿Y si pudiéramos crear un error que se llame `EdadInvalidaError`? Exactamente eso vamos a hacer.

## Momento de reflexión

¿Cuándo conviene crear una excepción propia en lugar de usar un simple `if` con `print`?

Cuando el error debe **detener** la ejecución

No tiene sentido continuar si el dato no es válido

Cuando el error puede ocurrir en **múltiples puntos**

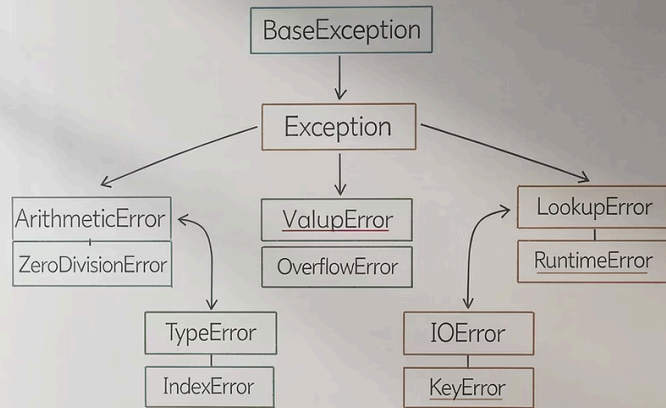
Y necesitas capturarlo siempre de la misma manera

Cuando quieres que el error tenga **un nombre propio**

Para que el código sea más legible y descriptivo



## Python Exception Hierarchy



# ¿Qué es una excepción en Python?

Python ya tiene muchas excepciones integradas: `ValueError`, `TypeError`, `ZeroDivisionError`... Todas son **clases** que heredan de `Exception`.



## Errores de Python

Son excepciones que Python lanza automáticamente cuando algo va mal, como dividir entre cero o pasar el tipo equivocado.



## Errores personalizados

Son excepciones que **nosotros definimos** para representar errores específicos de nuestro problema o aplicación.

# Definir tu primera excepción personalizada

Para crear una excepción propia, solo necesitas una clase que herede de `Exception`. No hace falta saber mucho de orientada a objetos: piensa en ello como **darle un nombre propio a un tipo de error**.

```
class EdadInvalidaError(Exception):  
    pass
```

✅ ¡Eso es todo! Con estas dos líneas ya tienes tu propia excepción. La palabra clave `pass` significa que no añadimos nada extra: el nombre ya es suficientemente descriptivo.

Puedes leer esto como: *"Crea un nuevo tipo de error llamado `EdadInvalidaError` que se comporta como cualquier otra excepción de Python."*

# Lanzar la excepción con raise

Una vez definida, podemos **lanzarla** en el momento adecuado usando `raise`. Esto interrumpe el flujo del programa igual que un error de Python.

```
class EdadInvalidaError(Exception):  
    pass  
  
def registrar_edad(edad):  
    if edad < 0 or edad > 120:  
        raise EdadInvalidaError("La edad debe estar entre 0 y 120.")  
    return edad  
  
registrar_edad(-5)  
# EdadInvalidaError: La edad debe estar entre 0 y 120.
```

 `raise` funciona como una señal de alarma: detiene el programa y comunica exactamente **qué salió mal** y **dónde**.

# Reflexión: errores de Python vs. errores nuestros

¿Qué diferencia hay entre un `ValueError` que lanza Python y un `EdadInvalidaError` que lanzamos nosotros?

## Error de Python (`ValueError`)

- Lo genera Python automáticamente
- Describe un problema técnico general
- No dice nada sobre *tu* lógica de negocio

## Error personalizado (`EdadInvalidaError`)

- Lo lanzas tú en el momento preciso
- Describe un problema **específico de tu programa**
- Hace el código mucho más legible y mantenible



# Capturar la excepción con try-except

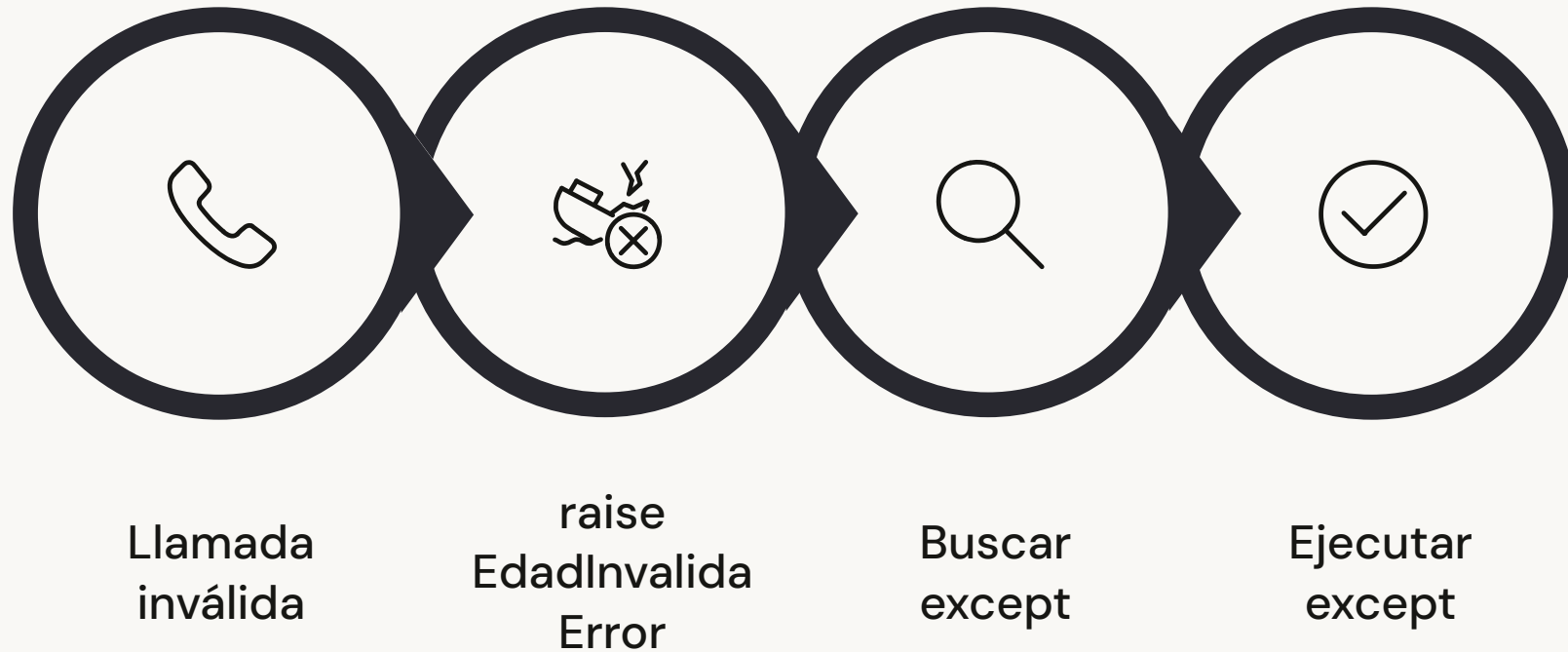
Ahora que sabemos lanzar nuestra excepción, podemos **capturarla** exactamente igual que cualquier otra excepción de Python. El nombre personalizado nos permite distinguirla de otros errores.

```
class EdadInvalidaError(Exception):  
    pass  
  
def registrar_edad(edad):  
    if edad < 0 or edad > 120:  
        raise EdadInvalidaError("La edad debe estar entre 0 y 120.")  
    return edad  
  
try:  
    registrar_edad(-5)  
except EdadInvalidaError as e:  
    print(f"Error de validación: {e}")  
except ValueError:  
    print("Por favor, introduce un número entero.")
```

✅ Fíjate: podemos capturar `EdadInvalidaError` por separado de `ValueError`, lo que nos da un control mucho más preciso sobre los errores.

# El flujo completo de una excepción personalizada

Veamos paso a paso cómo se mueve el control del programa cuando usamos una excepción personalizada.



Este flujo garantiza que los errores se gestionen de forma ordenada y predecible, sin que el programa se rompa de manera inesperada.

# Integración completa: validación en bucle

En la práctica, es muy habitual pedir datos al usuario en un bucle `while` hasta que introduzca algo válido. Las excepciones personalizadas encajan perfectamente en este patrón.

```
class EdadInvalidaError(Exception):
    pass

def validar_edad(edad):
    if edad < 0 or edad > 120:
        raise EdadInvalidaError("Edad fuera de rango (0-120).")
    return edad

while True:
    try:
        entrada = int(input("Introduce tu edad: "))
        edad = validar_edad(entrada)
        print(f"Edad registrada correctamente: {edad}")
        break
    except EdadInvalidaError as e:
        print(f"Error de validación: {e}")
    except ValueError:
        print("Por favor, introduce un número entero válido.")
```



# Actividad práctica

Es tu turno. Vamos a poner en práctica todo lo aprendido con un ejercicio guiado.

## Define tu excepción

- 1 Crea una clase `PrecioInvalidoError` que herede de `Exception`. Un precio válido debe ser mayor que 0 y menor que \$10.000.

## Lánzala con `raise`

- 2 Escribe una función `validar_precio(precio)` que lance `PrecioInvalidoError` con un mensaje descriptivo si el precio no es válido.

## Captúrala con `try-except`

- 3 Usa un bucle `while` para pedir el precio al usuario y captura tanto `PrecioInvalidoError` como `ValueError` con mensajes diferentes.



# Solución orientativa de la actividad

Aquí tienes una posible solución. Compara con la tuya y observa las diferencias.

```
class PrecioInvalidoError(Exception):
    pass

def validar_precio(precio):
    if precio <= 0 or precio >= 10000:
        raise PrecioInvalidoError(
            "El precio debe estar entre 0 y 10.000 €."
        )
    return precio

while True:
    try:
        entrada = float(input("Introduce el precio (€): "))
        precio = validar_precio(entrada)
        print(f"Precio registrado correctamente: {precio:.2f} €")
        break
    except PrecioInvalidoError as e:
        print(f"Error de validación: {e}")
    except ValueError:
        print("Por favor, introduce un número válido.")
```

# Múltiples excepciones en un proyecto real

Cuando el programa crece, es habitual definir **varias excepciones personalizadas**, una por cada tipo de error del dominio. Esto hace el código mucho más organizado y fácil de depurar.

```
# Jerarquía de excepciones para un sistema de habitaciones
```

```
class HotelError(Exception):
```

```
    """Excepción base del sistema hotelero."""
```

```
    pass
```

```
class HabitacionNoDisponibleError(HotelError):
```

```
    pass
```

```
class ReservaInvalidaError(HotelError):
```

```
    pass
```

```
class ClienteNoEncontradoError(HotelError):
```

```
    pass
```

- ✓ Todas heredan de `HotelError`, que a su vez hereda de `Exception`. Esto permite capturar cualquier error del sistema con un solo `except HotelError`, o ser muy específico con cada subtipo.



# Caso de uso: sistema de gestión de habitaciones

Veamos cómo las excepciones personalizadas mejoran un sistema real de reservas de hotel.



## HabitacionNoDisponibleError

Se lanza al intentar reservar una habitación que ya está ocupada. El sistema puede sugerir alternativas.



## ReservaInvalidaError

Se lanza cuando las fechas de entrada y salida no son coherentes, por ejemplo, si la salida es anterior a la entrada.



## ClienteNoEncontradoError

Se lanza cuando se intenta modificar o consultar una reserva de un cliente que no existe en el sistema.

# Buenas prácticas al crear excepciones



## Nombres descriptivos

El nombre debe explicar qué salió mal. Usa el sufijo Error siempre: `EdadInvalidaError`, no `MiError`.



## Hereda de Exception

Nunca heredes directamente de `BaseException`. `Exception` es la base correcta para errores de aplicación.



## Mensajes claros

Incluye siempre un mensaje que explique el problema y, si es posible, el valor que lo causó.



## No abuses de ellas

Crea excepciones personalizadas solo cuando el error tiene un significado propio en tu dominio. No las uses para todo.